

Initia Router API

Security Assessment

July 15th, 2025 — Prepared by OtterSec

James Wang james.wang@osec.io

Yordan Stoychev anatomic@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope 2

Findings **3**

General Findings **4**

OS-IRA-SUG-00 | Code Refactoring 5

OS-IRA-SUG-01 | Code Maturity 6

Vulnerability Rating Scale **7**

Procedure **8**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Initia engaged OtterSec to assess the `router-api` program. This assessment was conducted between June 11th and June 19th, 2025. For more information on our auditing methodology, refer to chapter 05.

Key Findings

We produced 2 findings throughout this audit engagement.

In particular, we recommended codebase improvements to enhance correctness, ensure adherence to coding best practices (OS-IRA-SUG-01), and advised updating the codebase to improve functionality and mitigate potential security issues (OS-IRA-SUG-00).

Scope

The source code was delivered to us in a Git repository at https://github.com/initia-labs/router-api. This audit was performed against 50dc18d.

A brief description of the program is as follows:

Name	Description
router-api	It acts as a middleware service for composing and transforming cross-chain token transfer routes.

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Findings

Overall, we reported 2 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	0
INFO	2

© 2025 Otter Audits LLC. All Rights Reserved.

03 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-IRA-SUG-00	Recommendation for updating the codebase to improve functionality and mitigate potential security issues.
OS-IRA-SUG-01	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-IRA-SUG-00

Description

1. The current logic in `evm::toDestinationAddressDataEvm` utilizes string replacement to left-pad EVM addresses, assuming they are always 20 bytes. This approach is fragile and fails for 32-byte addresses (such as Cosmos module accounts). A more robust solution is to pad addresses to 32 bytes, ensuring correct behavior across all valid address lengths.

   >_ *src/app/transfer/external/_utils/lz/evm.ts* RUST

   ```
   function toDestinationAddressDataEvm(
     destinationAddress: string,
     composeData?: string,
   ) {
     // if composeData is provided, destinationAddress is the l1ForwardAddress
     return composeData
       ? destinationAddress
       : destinationAddress.replace("0x", "0x000000000000000000000000");
   }
   ```

2. `index::createTxWithTargetErc20ToLocalMsg` assumes there is only one message in the transaction but does not explicitly validate it, which may result in incorrect behavior if multiple messages are present. A sanity check should be added to enforce this constraint.

Remediation

Incorporate the above-stated refactors.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-IRA-SUG-01

Description

1. Hardcoding token decimals to 6 or 18 in `msg.ts`, `memo.ts`, and `erc20.service.ts` is fragile, as not all tokens adhere to this standard. This may result in incorrect conversions or failed transfers for tokens with non-standard decimals. Although the current config assets utilize 6/18 decimals, it is more robust to fetch decimals dynamically from the token source.
2. The current implementation updates `denom` fields only for certain operation types, such as `transfer` or `op_init_transfer`, assuming other types will appear in certain routes. This assumption is fragile and may break silently if new wrappers are added. Although no skip routes to L2s via EVM currently exist, it is safer to explicitly raise an exception to guard against future changes.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.